Actividades Práctico-Experimentales

Nro. 004

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	1: Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	Implementa algoritmos de abstracción mediante el diseño lógico y físico de Tipos de Datos Abstractos (TDA), asegurando eficiencia en la gestión de memoria.
Práctica Nro.	004
Título de la Práctica	Listas Dinámicas Simples sin Overhead y su Impacto en la CPU.
Nombre del Docente	Cristian Ramiro Narváez Guillén
Integrantes del Grupo 09	Isaac Vire Axel Jimenez Danny Beltran Noelia Bustan Lucas Orellana
Fecha	jueves 22 y viernes 23 de octubre 2026
Horario	Jueves 22 de 10:30 – 11:30 Viernes 23 de mayo 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- **Diseñar e implementar** el Tipo de Dato Abstracto (TDA) Lista Simple Enlazada mediante la instanciación dinámica de nodos en JavaScript, evitando el uso de arreglos nativos.
- **Cuantificar y contrastar** el tiempo de ejecución T(n) entre la inserción de elementos al inicio de un arreglo dinámico (operación *unshift*) versus la inserción en la cabeza de una lista enlazada (operación *prepend*).
- **Evaluar** los principios de *Green Computing*, comprendiendo cómo la reducción de la complejidad temporal de ***O(n)*** a ***O(1)*** disminuye los ciclos de reloj de la CPU, mitigando el consumo energético y térmico del hardware.

3. Materiales y reactivos:

- Entorno de ejecución Node.js (versión LTS 20.x o superior) instalado.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación de estructuras dinámicas.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Se aplica ABP para resolver un problema de consistencia de datos en la transferencia de pacientes entre médicos.

Contexto para el estudiante: *Imagina que trabajas para el equipo de ingeniería de una nueva red social. El servidor debe procesar millones de peticiones de "siguiente video" por segundo. Si usas una estructura ineficiente, el servidor consumirá más electricidad, aumentará su temperatura y elevará los costos operativos.*

*Para resolver esto, implementaremos una **Lista Doblemente Enlazada** que permita saltar entre videos hacia adelante y hacia atrás sin tener que recorrer toda la lista desde el principio.*

Tarea 1: Creación del TDA Nodo y Lista Simple
Empezaremos construyendo la unidad básica de memoria dinámica: el Nodo.

1. En su directorio de trabajo, cree un archivo llamado lab04_lista_simple.js.
2. Implemente las clases Nodo y ListaSatelital utilizando clases modernas de JavaScript (ES6).

```
// Tarea 1: Implementación del TDA Lista Enlazada
class Nodo {
        constructor(dato) {
        this.dato = dato; // Carga útil (la coordenada)
        this.siguiente = null; // Puntero al siguiente nodo (referencia)
        }
}

class ListaSatelital {
        constructor() {
        this.cabeza = null; // Primer elemento de la lista
        this.tamaño = 0;// Contador para S(n)
        }

        // Método para insertar al inicio: Complejidad O(1)
        prepend(dato) {
        const nuevoNodo = new Nodo(dato);
        if (!this.cabeza) {
        this.cabeza = nuevoNodo;
        } else {
        nuevoNodo.siguiente = this.cabeza;
        this.cabeza = nuevoNodo;
        }
        this.tamaño++;
        }
}

module.exports = { ListaSatelital };
```

Tarea 2: Simulación del Estrés Térmico (Arreglo Nativo vs Lista Enlazada)

Vamos a demostrar empíricamente por qué la Lista Enlazada es una estructura de datos sostenible (Green Computing) frente a inserciones masivas.

1. Cree un archivo llamado lab04_benchmark.js e importe la clase creada.

2. Implemente un script que mida los milisegundos requeridos para insertar N = 100,000$ coordenadas al inicio de ambas estructuras.

```
const { performance } = require('perf_hooks');
const { ListaSatelital } = require('./lab04_lista_simple');

const N = 100000; // 100 mil registros satelitales

console.log(`[Benchmark] Iniciando inserción de ${N} elementos al INICIO...`);

// ------------------------------------------------------------
// PRUEBA 1: ARREGLO NATIVO (Desplazamiento de memoria O(n))
// ------------------------------------------------------------
let arregloHistorial = [];
const startArray = performance.now();

for (let i = 0; i < N; i++) {
        // unshift() fuerza a V8 a mover los índices de todos los elementos previos
        arregloHistorial.unshift({ id: i, lat: 10.5, lng: -79.2 });
}

const endArray = performance.now();
const timeArray = (endArray - startArray).toFixed(2);
console.log(`[Energía Alta] Arreglo nativo (unshift): ${timeArray} ms`);

// ------------------------------------------------------------
// PRUEBA 2: LISTA ENLAZADA (Asignación de punteros O(1))
// ------------------------------------------------------------
let listaHistorial = new ListaSatelital();
const startList = performance.now();

for (let i = 0; i < N; i++) {
        // prepend() solo reasigna un puntero, sin mover elementos existentes
        listaHistorial.prepend({ id: i, lat: 10.5, lng: -79.2 });
}

const endList = performance.now();
const timeList = (endList - startList).toFixed(2);
console.log(`[Energía Baja] Lista Enlazada (prepend): ${timeList} ms`);

// Conclusión de impacto
const multiplicador = (timeArray / timeList).toFixed(1);
console.log(`\n[Auditoría Verde] La lista enlazada es ${multiplicador} veces más rápida.`);
console.log(`Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.`);
```

3. Ejecute el script con node lab04_benchmark.js.
4. **Análisis:** Documente los tiempos arrojados. Notará que la diferencia es abismal. Mientras que el arreglo presenta una degradación asintótica $O(n^2)$ en el bucle completo, la lista mantiene un crecimiento lineal $O(n)$.

Tarea 3: Implementación del Recorrido (Traversal)

A diferencia de un arreglo, no podemos acceder a un nodo de la lista mediante índices (ej. lista[5]). Debemos recorrer la lista.

1. Regrese al archivo lab04_lista_simple.js y agregue el siguiente método dentro de la clase ListaSatelital.

```
// Método para imprimir la lista (Recorrido secuencial O(n))
imprimirPrimeros(limite) {
let actual = this.cabeza;
let contador = 0;
let salida = "";

while (actual && contador < limite) {
salida += `[ID: ${actual.dato.id}] -> `;
actual = actual.siguiente;
contador++;
}
console.log(salida + "NULL");
}
```

2. Pruebe el método en su archivo benchmark.js añadiendo al final: listaHistorial.imprimirPrimeros(5); para verificar la correcta vinculación de los nodos.

6. Resultados Esperados

Resultados de la Tarea 1

```
class Nodo {
    constructor(dato) {
        this.dato = dato; // Carga útil (la coordenada en si)
        this.siguiente = null; // Puntero al siguiente nodo (referencia que conecta los nodos)
    }
}
class ListaSatelital {
    constructor() {
        this.cabeza = null; // Primer elemento de la lista
        this.tamaño = 0;    // Contador para S(n), cantidad de nodos almacenados
    }
    // Método para insertar al inicio para generar la Complejidad O(1), solo se reasignan referencias
    prepend(dato) {
        const nuevoNodo = new Nodo(dato);
        if (!this.cabeza) {
            this.cabeza = nuevoNodo;
        } else {
            nuevoNodo.siguiente = this.cabeza;
            this.cabeza = nuevoNodo;
        }
        this.tamaño++;
    }
    // Método para imprimir la lista enlazada (Recorrido secuencial O(n))
    imprimirPrimeros(limite) {
        let actual = this.cabeza;
        let contador = 0;
        let salida = "";
        while (actual && contador < limite) {
            salida += `[ID: ${actual.dato.id}] -> `;
            actual = actual.siguiente;
            contador++;
        }
        console.log(salida + "NULL");
    }
}
module.exports = { ListaSatelital };
```

Resultados de la Tarea 2

```
1   const { performance } = require('perf_hooks');
2   const { ListaSatelital } = require('./lab04_lista_simple');
3
4   const N = 100000; // 100 mil registros satelitales
5
6   console.log(`[Benchmark] Iniciando inserción de ${N} elementos al INICIO...`);
7
8   // ------------------------------------------------------
9   // PRUEBA 1: ARREGLO NATIVO (O(n) por inserción y O(n²) en el bucle completo)
10  // ------------------------------------------------------
11  let arregloHistorial = [];
12  const startArray = performance.now();
13
14  for (let i = 0; i < N; i++) {
15      // unshift() fuerza a V8 a mover los índices de todos los elementos previos, mayor latencia generada
16      arregloHistorial.unshift({ id: i, lat: 10.5, lng: -79.2 });
17  }
18
19  const endArray = performance.now();
20  const timeArray = (endArray - startArray).toFixed(2);
21  console.log(`[Energía Alta] Arreglo nativo (unshift): ${timeArray} ms`);
22
```

```
// ------------------------------------------------------------
// PRUEBA 2: LISTA ENLAZADA (Asignación de punteros O(1))
// ------------------------------------------------------------
let listaHistorial = new ListaSatelital();
const startList = performance.now();

for (let i = 0; i < N; i++) {
    // prepend() solo reasigna dos referencias, sin mover ningún nodo existente
    listaHistorial.prepend({ id: i, lat: 10.5, lng: -79.2 });
}

const endList = performance.now();
const timeList = (endList - startList).toFixed(2);
console.log(`[Energía Baja] Lista Enlazada (prepend): ${timeList} ms`);

// Conclusión de impacto por el uso de ambos métodos
const multiplicador = (timeArray / timeList).toFixed(1);
console.log(`\n[Auditoría Verde] La lista enlazada es ${multiplicador} veces más rápida.`);
console.log(`Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.`);
listaHistorial.imprimirPrimeros(5);
```

```
[Benchmark] Iniciando inserción de 100000 elementos al INICIO...
[Energía Alta] Arreglo nativo (unshift): 4003.60 ms
[Energía Baja] Lista Enlazada (prepend): 11.92 ms

[Auditoría Verde] La lista enlazada es 335.9 veces más rápida.
Ahorro directo de ciclos de CPU y consumo eléctrico en el servidor.
[ID: 99999] -> [ID: 99998] -> [ID: 99997] -> [ID: 99996] -> [ID: 99995] -> NULL
```

Análisis de los resultados de la Tarea 2

En la Tarea 2 se compara la inserción de 100,000 coordenadas al inicio de un arreglo y de una lista enlazada. La diferencia entre los resultados se debe al modo de inserción. En el arreglo, cada unshift obliga a desplazar una posición todos los elementos ya almacenados, por lo que la k-ésima inserción cuesta O(k). Al sumar el costo de todas las inserciones del bucle, $0 + 1 + 2 + \ldots + (n-1)$, se llevan a cabo $n(n-1)/2$ operaciones, por lo que la complejidad al final es $O(n^2)$. En la lista enlazada, prepend solo asigna dos nuevas referencias, con el uso de nuevoNodo.siguiente y this.cabeza conectando nodos sin mover ninguno ya presente, por lo que cada inserción es O(1) y el bucle completo resulta en O(n). Ambas estructuras son O(n) en cuanto al espacio que ocupan, aunque la lista consume más RAM por elemento al almacenar un puntero en cada nodo. A cambio de la memoria consumida, se obtienen menos ciclos de CPU, lo que reduce el consumo energético y térmico. A manera de evidencia, en la ejecución, el arreglo tardó **4003.60 ms** y la lista **11.92 ms**, es decir, la lista fue 335.9 **veces** más rápida.

7. Preguntas de Control:

Responda fundamentando sus argumentos bajo una perspectiva lógica, algorítmica y filosófica:

1. **A diferencia de un arreglo que pre-reserva una cantidad específica de Bytes contiguos, la Lista Enlazada nace como una referencia nula (null). Físicamente en la RAM, ¿de qué está**

hecha la "conexión" (el campo this.siguiente) que une a dos nodos esparcidos aleatoriamente por el *Heap*?

Básicamente está formada por un conjunto de de bytes que almacenan un valor numérico que corresponde a una dirección de memoria lógica o virtual la cual señala la posición exacta dentro del Heap donde fue reservado el siguiente nodo, en motores como en Java V8, esta referencia utiliza etiquetado de punteros para empaquetar esos mismos bits en la misma dirección de memoria junto con marcas que identifican el tipo de valor y al momento de ejecutar la lectura de "this.siguiente", la MMU (Unidad de Manejo de Memoria) y las tablas de páginas del sistema operativo traducen esa dirección virtual a las celdas y posiciones físicas reales de los chips de RAM.

2. Al invocar unshift() en un arreglo, el motor V8 es la causa eficiente que desplaza la memoria. En la operación prepend() de la Lista Enlazada, ¿cuál es el único agente o variable que se debe modificar para lograr la inserción exitosa sin desplazar datos?

En la operación prepend(), la variable principal que se actualiza es la referencia de la cabeza (this.cabeza), que apunta al primer nodo de la lista enlazada.

Para insertar un nuevo nodo sin desplazar los datos existentes en memoria, se realizan dos reasignaciones de referencias:

1. Se conecta el nuevo nodo con el primero de la lista mediante nuevoNodo.siguiente = this.cabeza.
2. Se actualiza la cabeza para que apunte al nuevo nodo mediante this.cabeza = nuevoNodo.

A diferencia de unshift(), el método de los arreglos nativos de JavaScript que inserta elementos al inicio y puede requerir reorganizar los índices existentes, prepend() modifica las referencias de la lista enlazada sin desplazar sus nodos. Por ello, la inserción tiene una complejidad temporal O(1).

3. Si los arreglos nativos de JavaScript modernos están sumamente optimizados por lenguajes de bajo nivel (C++), ¿cuál es el verdadero telos (propósito final) de tomarse el tiempo para construir manualmente una estructura como la Lista Enlazada en entornos de producción?

Aunque los arreglos nativos en JavaScript son muy eficientes para la mayoría de tareas cotidianas, las listas enlazadas resultan fundamentales en sistemas de alto rendimiento porque permiten modificar, organizar y procesar grandes volúmenes de datos de forma inmediata y sin congelar la aplicación. A diferencia de los arreglos tradicionales, donde quitar o mover información al inicio requiere reordenar todo el contenido, una lista enlazada permite insertar o eliminar elementos al instante y pausar procesos complejos para reanudarlos después sin perder el hilo, garantizando así que programas exigentes o plataformas web masivas funcionen con fluidez y rapidez.

4. Un nodo que pierde la referencia (es decir, el nodo anterior ya no apunta a él) sigue existiendo físicamente en los transistores de la memoria RAM con su carga útil intacta, pero la Lista ya no lo reconoce. ¿Ha perdido ese nodo su esencia computacional? ¿Qué sucederá eventualmente con su existencia cuando despierte el *Garbage Collector*?

Su esencia computacional, desde el punto de la lista enlazada, se ha perdido porque ya no es útil de esa manera, no se puede hacer nada con él de manera lógica si no se lo reconoce, en ese aspecto

ahora realmente solo es una carga en la memoria RAM. Pero realmente esto cambia si lo vemos desde la perspectiva de la memoria, desapareció de la lista, pero sigue disponible si se lo referencia desde otra parte del código, lo que podría dar pie a problemas de seguridad antes de que el Garbage Collector se accione. Pero más allá de eso, lo único que pasará a partir de ahora será que el Garbage Collector se dará cuenta de que no es alcanzable y lo borrará.

8. Bibliografía

Liste las fuentes utilizadas. Use normas **IEEE**. Pueden ser libros, manuales, artículos, tutoriales académicos, documentación oficial de software.

[1] B. Goetz, Java Concurrency in Practice. Addison-Wesley, 2022.

[2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.

[3] J. Bloch, Effective Java, 3rd ed. Boston, MA: Addison-Wesley, 2018.

9. Anexos

Enlace al GitHub: https://github.com/B3kk4r/Estructura-de-datos-Grupo9.git